RestOps / MED Restaurant SaaS

Azure Migration Strategy and Execution Plan

Audience: CEO, board, product leadership, engineering leadership

Prepared by: Enterprise Architecture

Date: July 3, 2026

Decision Required: Approve target Azure architecture, migration funding, and phased execution plan.

---

1. Executive Summary

RestOps / MED is currently implemented as a React/Vite restaurant SaaS platform with Supabase-backed data, storage, authentication, edge functions, realtime events, and PostgreSQL migrations. The platform already contains the core product domains required for restaurant operations: invoices, payments, inventory, products, vendors, auto-ordering, recipes, labor, SmartPrep, reporting, billing, platform administration, integrations, and AI workflows.

The proposed migration moves RestOps to a standardized Azure SaaS architecture that keeps the application/control plane shared while giving **every restaurant client the same isolated data plane**:

- Dedicated Azure Resource Group per client.
- Dedicated Azure Blob Storage per client for invoices, receipts, exports, and document-heavy workloads.
- Dedicated Azure PostgreSQL Flexible Server per client for invoices, payments, products, inventory, ledgers, POS data, audit logs, and operational records.
- Shared Azure application plane for frontend, APIs, workers, orchestration, notifications, CI/CD, governance, and tenant routing.
- Azure Service Bus event backbone for email, notifications, messages, webhooks, invoice processing, and other event-driven workflows.
- Terraform-driven provisioning so every client receives the same repeatable, governed architecture.

This approach is intentionally more isolated than a traditional shared-database SaaS model. It is the correct fit for the stated business requirement: predictable client isolation, heavy invoice/file upload support, per-client storage/database usage visibility, independent scaling, and enterprise credibility.

---

2. Current Platform Architecture

2.1 Current Repo Facts

The current codebase contains:

Area	Current Repo Evidence
Registered application routes	51 routes in `code/src/pages.config.js`
Module page files	71 page components under `code/src/modules`
Supabase edge functions	40 functions under `code/supabase/functions`
Database migrations	251 SQL migration files under `code/supabase/migrations`
Tenant scoping model	`organization_id`, `brand_id`, `location_id` in `code/src/lib/apiClient.js`

2.2 Current Application Modules

The current React application includes these major functional domains:

- Dashboard and executive reporting.
- Invoice intake, AI extraction, approval, allocations, and AP workflow.
- Payments, payment accounts, payment verification, and billing.
- Products, inventory, wastage, AvT costing, auto-ordering, and receiving workflows.
- Vendors, vendor item mapping, vendor bidding, and vendor onboarding.
- Recipes, menu engineering, digital menu, delivery aggregation, KDS, and online ordering.
- Labor, labor schedules, payroll export, tip pooling, shift board, and time clock.
- SmartPrep, AI insights, custom reports, performance, CRM, food safety, commissary.
- Platform admin, platform organizations, platform users, platform plans, audit logs, and billing operations.
- Integrations and developer portal.

2.3 Current Runtime Model

Current architecture is primarily:

```
React/Vite SPA
  -> Supabase Auth
  -> Supabase PostgreSQL
  -> Supabase Storage
  -> Supabase Edge Functions
  -> External systems: Stripe, Dwolla, Checkbook, POS, SendGrid, Gemini, QuickBooks
```

Current Supabase edge functions include:

- Invoice and document workflows: `invoice-processing`, `process-email-invoices`.
- Payments and billing: `stripe-webhook`, `create-checkout-session`, `create-payment-intent`, `create-portal-session`, `process-payout`, `payout-webhook`, `process-checkbook-payout`, `checkbook-webhook`, `billing-worker`.

- POS and operational integrations: `pos-webhook`, `pos-sync`, `sync-delivery-menus`, `sync-accounting`.
- AI workflows: `ai-insights-chat`, `smartprep-cron`, `generate-prep-sheet`, `forecast-labor`, `evaluate-vendor-bids`, `voice-copilot-parser`.
- Platform workflows: `invite-user`, `invite-client`, `vendor-onboarding`, `process-onboarding`, `webhook-dispatcher`, `team-worker`, `schedule-reports`, `dashboard-report-scheduler`.
- Infrastructure/ops: `api-gateway`, `pg-backup`, `create-api-key`, `create-webhook-endpoint`, `iot-ingest`, `iot-webhook`, `notify-demo-request`, `process-marketing`, `calculate-royalties`.

2.4 Current Architecture Strengths

- Product modules are already separated by business capability.
- Tenant context exists in the application model through organization, brand, and location.
- The database migration footprint is mature and extensive.
- The system already has event-driven behavior through edge functions, webhooks, scheduled functions, and database triggers.
- AI workflows are already identified and separated into functions/workers.
- The product domain is operationally broad enough to justify an enterprise cloud architecture.

2.5 Current Architecture Limitations

- Supabase-centric architecture limits control over dedicated per-client infrastructure.
- File-heavy invoice workloads are not isolated per client by default.
- Database-heavy clients can create noisy-neighbor risk in a shared database model.
- Edge function workflows need a stronger queue, retry, dead-letter, and observability model.
- Tenant routing and resource mapping must become explicit before per-client data planes can scale.
- Production-grade secrets, private endpoints, cost metering, and infrastructure governance need Azure-native implementation.

---

3. Target Azure Architecture

3.1 Architectural Principle

The final design uses **one standard architecture for every client**.

It is not a custom design per client. It is a repeatable architecture template:

```
Shared RestOps Application / Control Plane
  +
Standard Per-Client Isolated Data Plane
```

Every restaurant client receives the same standard resource pattern:

```
rg-restops-client-{client_id}-{env}
  - Dedicated Blob Storage
  - Dedicated PostgreSQL Flexible Server
  - Client Key Vault references or client-specific keys
  - Private endpoints
  - Diagnostic settings
  - Budget alerts
  - Usage metering
```

3.2 Target Architecture Diagram

Primary image artifact:

`architecture/restops_repo_aligned_standard_architecture.svg`

This diagram shows:

- Shared application and automation plane.
- Repo-specific application modules and functions.
- Event backbone.
- Tenant registry.
- Standard isolated client data plane.
- Dedicated storage and PostgreSQL per client.
- Usage/cost visibility.

3.3 Shared Application and Control Plane

The shared platform plane is operated centrally and reused by every client.

Capability	Azure Target	Purpose
Frontend hosting	Azure Static Web Apps	Hosts React/Vite SPA with global CDN and TLS.
Global ingress	Azure Front Door + WAF	Edge routing, TLS, WAF, global protection.
API gateway	Azure API Management	JWT validation, throttling, API policies, versioning, CORS.
Core application API	Azure Container Apps	Runs RestOps domain services and tenant routing logic.
Webhook receivers	Azure Functions	Handles Stripe, Dwolla, Checkbook, POS, IoT, and external callbacks.
Async workers	Azure Functions or Container Apps Jobs	Processes invoice extraction, notifications, reports, sync jobs.
Event backbone	Azure Service Bus	Durable event-driven workflows, fan-out, retries, DLQ.
Control database	Azure PostgreSQL Flexible Server or Azure SQL	Tenant registry, plans, provisioning status, routing endpoints, usage summaries.
Secrets	Azure Key Vault	Secrets, API keys, webhook secrets, DB credentials, encryption keys.
Identity	Microsoft Entra External ID	Auth, MFA, SSO, token claims, external tenant users.
Observability	Application Insights + Log Analytics	Traces, metrics, dependency telemetry, alerts, SLOs.
IaC	Terraform	Repeatable provisioning for shared and client resources.

3.4 Standard Per-Client Data Plane

Each client receives an identical resource group architecture.

Component	Purpose
Dedicated Resource Group	Logical and operational isolation per client.
Dedicated Blob Storage	Stores invoices, receipts, exports, reports, vendor documents.
Dedicated PostgreSQL Flexible Server	Stores client operational tables: invoices, payments, products, inventory, recipes, vendors, audit logs, POS, ledgers.
Private Endpoints	Keeps database and storage off the public internet where feasible.
Client-scoped Key Vault keys	Supports per-client encryption policy and enterprise requirements.
Diagnostic Settings	Push metrics/logs into central monitoring.
Budget Alert	Tracks and alerts on per-client cloud spend.
Usage Metering	Tracks invoice count, blob bytes, DB size, row counts, event counts, estimated cost.

3.5 Why This Architecture Is Appropriate

This model solves the central business concern: some clients may upload very large numbers of invoices and generate significant database and storage growth.

With dedicated Blob Storage:

- File growth is isolated.
- Per-client storage usage is easy to track.
- Lifecycle policies can be enforced per client.
- Storage cost can be reported per client.

With dedicated PostgreSQL:

- Heavy invoice/payment tables are isolated.
- Large clients cannot degrade other clients.
- Backup/restore can be done per client.
- Database storage and performance metrics are visible per client.
- Scaling decisions are independent.

With shared application/control plane:

- Engineering operates one application.
- Clients all receive the same standard platform.
- CI/CD remains centralized.
- Business logic remains consistent.
- No client-specific code forks are required.

---

4. Azure Replacement Mapping

Current Service / Pattern	Azure Replacement	Notes
React/Vite SPA	Azure Static Web Apps	Same SPA build, Azure-hosted.
Supabase Auth	Microsoft Entra External ID	Enterprise-ready identity, MFA, SSO, token claims.
Browser-to-Supabase direct data access	API Management + Container Apps API	Moves data access behind a controlled API boundary.
Supabase PostgreSQL	Azure PostgreSQL Flexible Server per client	Dedicated DB server per client data plane.
Supabase Storage	Azure Blob Storage per client	Dedicated storage account/container strategy per client.
Supabase Edge Functions	Azure Functions / Container Apps workers	Webhooks, async jobs, scheduled jobs.
pg_net webhooks	Azure Service Bus / Event Grid	Durable event routing instead of direct DB-to-HTTP coupling.
pg_cron scheduled jobs	Azure Timer Functions	Scheduled SmartPrep, reports, backups, billing workers.
Supabase Realtime	Azure Web PubSub or SignalR	Live product/margin/notification updates if needed.
Supabase environment secrets	Azure Key Vault + Managed Identity	Centralized, auditable secrets management.
Supabase logs	Application Insights + Log Analytics	Central enterprise observability.
Supabase backups	PostgreSQL Flexible Server PITR/backups	Per-client restore and retention policies.

5. Control Plane Design

5.1 Control Plane Purpose

The control plane is the operational brain of the SaaS platform.

It does not hold client invoice files or client operational data. It holds metadata needed to route, provision, bill, monitor, and govern clients.

5.2 Control Plane Responsibilities

- Client onboarding and provisioning.
- Tenant registry and routing.

- Subscription and plan management.
- User and organization metadata.
- Client resource status.
- Resource endpoint registry.
- Usage and cost summary.
- Deployment status.
- Platform-level audit.
- Feature flags and entitlement rules.

5.3 Tenant Registry

Suggested core table:

```
tenant_registry
  id
  client_id
  organization_id
  tenant_slug
  environment
  status
  plan_id
  azure_subscription_id
  resource_group_name
  postgres_server_name
  postgres_database_name
  postgres_fqdn
  storage_account_name
  blob_endpoint
  key_vault_name
  service_bus_namespace
  provisioning_state
  schema_version
  created_at
  updated_at
```

5.4 Request Routing

Runtime request flow:

```
User JWT
  -> API Management validates token
  -> Container Apps API extracts organization/client claim
  -> API queries tenant_registry
  -> API opens connection to that client's PostgreSQL
  -> API uses that client's Blob Storage endpoint for documents
  -> Response returns to frontend
```

5.5 Control Plane Database

The control plane database should be small, highly governed, and independent from client data planes.

It should contain:

- Organizations.
- Plans.
- Provisioning jobs.
- Tenant registry.
- Usage daily summaries.
- Billing mapping.
- Feature entitlements.
- Platform audit logs.
- Schema migration status per client.

---

6. Terraform and Infrastructure Automation

6.1 Why Terraform Is Required

Because every client receives an identical isolated data plane, manual provisioning is not viable.

Terraform is required for:

- Consistent client resource groups.
- Repeatable storage/database/security configuration.
- Standard tags.
- Cost and budget policies.
- Private endpoint configuration.
- Diagnostic settings.
- Drift detection.
- Approval workflow before production changes.

6.2 Terraform Repository Structure

Recommended structure:

```
infra/
  modules/
    shared-platform/
      static-web-app/
      front-door-waf/
      api-management/
      container-apps/
      functions/
      service-bus/
      key-vault/
      monitoring/
      control-db/
    client-data-plane/
      resource-group/
      blob-storage/
      postgres-flexible-server/
      private-endpoints/
      diagnostics/
      budget-alerts/
      key-vault-access/
  envs/
    dev/
      shared.tfvars
    staging/
      shared.tfvars
    prod/
      shared.tfvars
  tenants/
    prod/
      client-template.tfvars
      client-{tenant_slug}.tfvars
```

6.3 Client Data Plane Terraform Module

The client module should create:

- Resource group.
- Storage account.
- Blob containers.
- PostgreSQL Flexible Server.
- PostgreSQL database.
- Firewall/private endpoint settings.
- Diagnostic settings.
- Budget alert.
- Tags.
- Key Vault access policies / RBAC assignments.
- Outputs written back to tenant registry.

6.4 Required Terraform Outputs

Each client provisioning run must output:

```
client_id
resource_group_name
storage_account_name
blob_endpoint
postgres_server_name
postgres_fqdn
postgres_database_name
key_vault_name
private_endpoint_ids
diagnostic_workspace_id
budget_id
```

These outputs are stored in the control plane tenant registry.

6.5 Provisioning Flow

```
Client onboarding request
  -> Client Portal writes provisioning request
  -> Provisioning Worker validates metadata
  -> Terraform plan generated
  -> Approval gate
  -> Terraform apply
  -> Azure resources created
  -> Database migrations applied
  -> Seed baseline data
  -> Tenant registry updated
  -> Client status = active
```

6.6 CI/CD

Recommended deployment pipeline:

```
Pull Request
  -> terraform fmt
  -> terraform validate
  -> security scan
  -> terraform plan
  -> architecture approval
  -> terraform apply
  -> smoke tests
  -> update tenant registry
```

Production changes require approval.

---

7. Event-Driven Architecture

7.1 Event Backbone

Use Azure Service Bus for business events.

Use Event Grid for Azure infrastructure events such as Blob-created events.

Do not use direct synchronous processing for webhooks, invoice extraction, emails, notifications, or report jobs.

7.2 Event Types

Initial event catalog:

Event	Producer	Consumers
`invoice.uploaded`	API / Blob event worker	invoice extraction worker
`invoice.extracted`	invoice worker	notification worker, audit worker
`invoice.approved`	API	AP workflow, payment scheduler, notification worker
`payment.initiated`	API	payment worker, audit worker
`payment.completed`	payment webhook worker	invoice update worker, notification worker, accounting sync
`pos.order.received`	POS webhook	POS sync worker, inventory/AvT worker
`product.price_changed`	invoice/variance worker	recipe costing, margin alerts
`inventory.low_stock`	inventory workflow	notification worker, auto-ordering worker
`report.requested`	dashboard/report API	report worker
`email.requested`	application workflows	email worker
`sms.requested`	application workflows	SMS worker
`webhook.delivery_requested`	platform integrations	webhook dispatcher

7.3 Standard Event Envelope

All events must use the same envelope:

```
{
  "event_id": "uuid",
  "event_type": "invoice.approved",
  "event_version": "1.0",
  "client_id": "client_123",
  "organization_id": "org_123",
  "brand_id": "brand_123",
  "location_id": "loc_123",
  "correlation_id": "trace_123",
  "idempotency_key": "invoice_123_approved",
  "occurred_at": "2026-07-03T12:00:00Z",
  "producer": "restops-core-api",
  "payload": {}
}
```

7.4 Queueing Pattern

Recommended pattern:

```
API transaction
  -> writes business data
  -> writes outbox_events row
  -> outbox dispatcher publishes Service Bus message
  -> worker consumes message
  -> worker updates client DB
  -> worker emits next event
  -> DLQ captures failures
```

7.5 Notification Workflow

```
Business event
  -> notification topic
  -> email subscription
  -> SMS subscription
  -> push subscription
  -> WhatsApp subscription
  -> audit subscription
```

This supports:

- Email notifications.
- SMS.
- WhatsApp/vendor messages.
- Mobile push.
- In-app notifications.
- Audit logging.

---

8. Data Architecture

8.1 Client PostgreSQL

Each client receives a dedicated PostgreSQL Flexible Server.

It stores:

- invoices
- invoice_line_items
- invoice_allocations
- payments
- ledger entries
- products
- inventory
- vendors
- recipes
- auto_orders
- POS data
- wastage logs
- audit logs
- notifications
- operational settings

8.2 Why Keep Tenant Columns Inside Dedicated DBs

Even with a dedicated client database, keep:

```
organization_id
brand_id
location_id
```

Reason:

- A client may have multiple brands.
- A client may have multiple locations.
- Franchise groups need hierarchy.
- Role access still needs brand/location boundaries.
- Current code already uses those scopes.

8.3 Database Migration Management

Because every client has a database, schema management must become a first-class platform capability.

Required table:

```
tenant_schema_versions
  client_id
  database_name
  current_schema_version
  target_schema_version
  last_migration_id
  migration_status
  last_success_at
  last_failure_at
  failure_message
```

Migration runner flow:

```
New migration merged
  -> CI validates migration
  -> staging client DB upgraded
  -> smoke tests pass
  -> production rollout begins
  -> apply migration client by client
  -> record result
  -> stop or quarantine on failure
```

8.4 Blob Storage

Each client receives dedicated Blob Storage.

Containers:

```
invoices
receipts
vendor-documents
exports
reports
attachments
```

Blob rules:

- Private by default.
- No public container access.
- Short-lived SAS upload/download URLs.
- Blob metadata includes client_id, organization_id, document_type.
- Encryption at rest.
- Optional customer-managed key.
- Lifecycle rules for archive/purge.
- Container/storage metrics recorded daily.

---

9. Security Architecture

9.1 Identity

Target identity service:

- Microsoft Entra External ID.
- MFA and conditional access where appropriate.
- JWT claims include role and tenant context.
- Optional enterprise SSO.

9.2 API Security

API Management enforces:

- JWT validation.
- Rate limits.
- Client/tenant claims required.
- CORS policy.
- Request size limits.
- API versioning.
- Request correlation ID.

9.3 Data Security

- Dedicated PostgreSQL per client.
- Dedicated Blob Storage per client.
- Private endpoints where feasible.
- Managed Identity for service-to-service access.
- Key Vault for secrets.
- No service credentials in frontend.
- Audit log for sensitive actions.

9.4 Authorization

Authorization remains layered:

```
Frontend role visibility
  + API authorization
  + tenant registry routing
  + PostgreSQL RLS / scoped queries
```

10. Usage Metering and Cost Tracking

10.1 Why Usage Metering Matters

The business requirement is clear: clients may upload huge invoice volumes. The platform must know which clients are driving storage, database, event, and AI cost.

10.2 Daily Usage Table

Recommended table:

```
tenant_usage_daily
  client_id
  organization_id
  usage_date
  invoice_count
  invoice_line_item_count
  blob_bytes
  blob_object_count
  postgres_allocated_bytes
  postgres_used_bytes
  event_count
  ai_extraction_count
  email_count
  sms_count
  estimated_storage_cost
  estimated_database_cost
  estimated_event_cost
  estimated_ai_cost
  total_estimated_cost
  created_at
```

10.3 Metrics Sources

Metric	Source
Blob size	Azure Storage metrics / inventory
Blob object count	Storage inventory
DB size	PostgreSQL metrics / SQL queries
Invoice count	Client PostgreSQL
Event count	Service Bus metrics
Email/SMS count	Worker logs / provider response
AI extraction count	Worker logs / invoice processing records
Cost	Azure Cost Management + internal allocation

11. Observability and Operations

11.1 Required Observability

Use:

- Application Insights.
- Log Analytics.
- Azure Monitor alerts.
- Azure Cost Management.
- Service Bus metrics.
- PostgreSQL metrics.
- Storage metrics.

11.2 Key Alerts

- Invoice extraction failure rate above threshold.
- Payment webhook failures.
- Service Bus DLQ message count greater than zero.
- Service Bus queue depth above threshold.
- Client DB storage above threshold.
- Client Blob Storage above threshold.
- API 5xx rate above threshold.
- API latency above SLO.
- Client budget threshold exceeded.
- Database CPU/memory/storage pressure.

11.3 Correlation ID

Every request and event must carry:

```
correlation_id
client_id
organization_id
user_id
event_id
```

This allows tracing from:

```
frontend click
  -> API request
  -> Service Bus event
  -> worker execution
  -> PostgreSQL write
  -> notification
```

---

12. Migration Plan

Phase 0: Architecture and Governance Baseline

Objective: finalize architecture, naming, tagging, Terraform structure, and security model.

Deliverables:

- Azure landing zone decisions.
- Resource naming standard.
- Tagging standard.
- Terraform module design.
- Tenant registry schema.
- Event envelope standard.
- Security and networking baseline.
- CI/CD approval model.

Exit criteria:

- CEO/CTO approval.
- Terraform repo initialized.
- Shared architecture diagram approved.

Phase 1: Shared Platform Foundation

Objective: create shared Azure foundation.

Build:

- Azure Front Door + WAF.
- Azure Static Web Apps.
- API Management.
- Container Apps environment.
- Azure Functions environment.
- Service Bus namespace.
- Key Vault.
- Application Insights.

- Log Analytics.
- Control Plane DB.

Exit criteria:

- React app deployed to Azure Static Web Apps.
- API Management can route to a test API.
- Service Bus test event processed.
- Logs visible in Application Insights.

Phase 2: Control Plane and Tenant Registry

Objective: build tenant routing and provisioning foundation.

Build:

- Tenant registry.
- Provisioning job table.
- Usage summary tables.
- Client provisioning workflow.
- Terraform output ingestion.
- Admin UI/API for provisioning status.

Exit criteria:

- New client request creates pending provisioning job.
- Terraform output updates tenant registry.
- API can resolve client data plane endpoint.

Phase 3: Standard Client Data Plane Module

Objective: create repeatable client resource group template.

Build Terraform module for:

- Client resource group.
- Client Blob Storage.
- Client PostgreSQL Flexible Server.
- Client database.
- Client private endpoints.
- Client diagnostic settings.
- Client budget alert.
- Client Key Vault references.

Exit criteria:

- One test client environment provisioned.
- Migrations applied successfully.

- Blob upload/download works through API.
- DB connection resolved through tenant registry.

Phase 4: API Migration

Objective: move data access from direct Supabase client pattern to API-managed access.

Migrate:

- Products.
- Invoices.
- Payments.
- Inventory.
- Vendors.
- Recipes.
- Dashboard queries.

Exit criteria:

- Core modules run through Container Apps API.
- Tenant routing works.
- API authorization works.
- Existing role behavior preserved.

Phase 5: File and Invoice Workflow Migration

Objective: move document-heavy workflows to client Blob Storage and Azure workers.

Migrate:

- Invoice upload.
- Email invoice import.
- Blob-created event.
- Invoice extraction worker.
- AI extraction status updates.
- Invoice review workflow.

Target flow:

```
Upload invoice
  -> client Blob Storage
  -> Event Grid
  -> Service Bus
  -> invoice-processing worker
  -> client PostgreSQL
  -> notification event
```

Exit criteria:

- Invoice upload works end to end.
- AI extraction writes to client DB.
- Failure writes DLQ and visible status.
- Large file upload tested.

Phase 6: Payment, POS, and Notification Events

Objective: migrate external events to Azure Functions and Service Bus.

Migrate:

- Stripe webhook.
- Dwolla webhook.
- Checkbook webhook.
- POS webhook.
- Notification events.
- Email/SMS/WhatsApp worker.

Exit criteria:

- Webhook signatures validated.
- Idempotency enforced.
- DLQ tested.
- Payment status update tested.
- Notification delivered from event.

Phase 7: Reporting, AI, and Scheduled Jobs

Objective: move scheduled and AI workloads.

Migrate:

- SmartPrep cron.
- AI insights.
- Labor forecast.
- Vendor bid evaluation.
- Dashboard reports.
- Scheduled reports.
- Accounting sync.

Exit criteria:

- Timer Functions running.
- AI calls tracked by client.
- Reports generated and stored in client Blob Storage.
- Usage metering captures AI/report cost drivers.

Phase 8: Observability, Cost, and Production Readiness

Objective: make the platform production-operable.

Build:

- Dashboards.
- SLOs.
- Alerts.
- Cost reporting.
- Usage metering jobs.
- Backup validation.
- Restore runbook.
- Incident response runbook.

Exit criteria:

- Per-client usage dashboard available.
- Cost alerts working.
- Backup/restore tested.
- Incident runbook approved.

Phase 9: Pilot and Cutover

Objective: migrate first live tenant safely.

Steps:

- Select pilot client.
- Provision client Azure data plane.
- Backfill data.
- Validate counts.
- Run parallel read mode.
- Run workflow smoke tests.
- Freeze writes briefly if required.
- Cut over routing.
- Monitor for 7 days.

Exit criteria:

- Pilot client stable.
- No data reconciliation gaps.
- Performance acceptable.
- Support runbook validated.

Phase 10: Rollout to All Clients

Objective: repeat migration using standard playbook.

Steps:

- Prioritize clients by risk and volume.
- Provision data planes.
- Migrate and validate data.
- Cut over client by client.
- Monitor cost and performance.
- Decommission old paths after stabilization.

Exit criteria:

- All active clients on Azure target architecture.
- Supabase runtime dependencies removed or explicitly retained by decision.
- Azure operating dashboards become source of truth.

---

13. Data Migration Approach

13.1 Migration Method

For each client:

```
Extract source data
  -> transform to target schema if required
  -> load client PostgreSQL
  -> copy files to client Blob Storage
  -> validate counts and checksums
  -> run application smoke tests
  -> cut over routing
```

13.2 Validation

Validate:

- invoice count
- invoice line item count
- payment count
- vendor count
- product count
- inventory count
- audit log count
- blob object count

- blob byte count
- sample file retrieval
- dashboard totals
- AP totals

13.3 Rollback

Rollback strategy:

- Preserve source environment until post-cutover window closes.
- Keep migration snapshots.
- Routing can point client back to old environment if cutover fails.
- Do not delete old data until reconciliation and retention approval.

---

14. Risks and Mitigations

Risk	Impact	Mitigation
Higher per-client infrastructure cost	Lower margin for small clients	Price plans accordingly; automate cost tracking; enforce budgets.
Many PostgreSQL servers to manage	Operational complexity	Terraform, migration runner, monitoring templates.
Schema upgrades across all clients	Migration risk	Tenant schema version table and phased rollout.
Event duplication	Duplicate emails/payments	Idempotency keys and unique constraints.
Queue backlog	Delayed workflows	Queue depth alerts and autoscaling workers.
Large invoice uploads	Storage cost and processing lag	Dedicated Blob Storage, lifecycle policies, async processing.
Secrets sprawl	Security exposure	Key Vault and Managed Identity.
Routing bugs	Client data access failure	Tenant registry validation and automated smoke tests.
Cutover data mismatch	Business disruption	Reconciliation reports and rollback window.

---

15. Governance Model

15.1 Architecture Review Board

Approves:

- Terraform module changes.
- Shared platform changes.
- Security baseline changes.
- Database migration policy.
- Client data plane template changes.

15.2 Change Management

Production change requirements:

- Pull request.
- Terraform plan.
- Security scan.
- Architecture approval for infrastructure changes.
- Deployment approval.
- Post-deployment smoke test.

15.3 Standards

Required standards:

- All resources tagged.
- All events include client_id and correlation_id.
- All secrets in Key Vault.
- No public database access.
- Blob Storage private by default.
- Per-client budget alert.
- Diagnostic settings enabled.
- Terraform manages infrastructure.

---

16. Success Metrics

Metric	Target
Client provisioning time	Under 30 minutes after approval
Invoice upload reliability	99.9% successful accepted uploads
Invoice extraction workflow	Retryable with DLQ for failures
Webhook processing	Idempotent, observable, retryable
Per-client cost visibility	Daily client-level cost estimate
Per-client usage visibility	Daily DB/storage/event usage
Restore readiness	Client restore runbook tested
Deployment repeatability	All client data planes from same Terraform module
Security posture	No public DB, no browser secrets, Key Vault-backed secrets

17. Recommended Executive Decision

Enterprise Architecture recommends approving the Azure migration using the following target model:

1. Shared application and control plane.
2. Standard isolated client data plane for every client.
3. Dedicated Blob Storage and PostgreSQL Flexible Server per client.
4. Azure Service Bus as the event backbone.
5. Terraform as the provisioning and governance standard.
6. Tenant registry as the core routing and operations source of truth.
7. Phased migration beginning with shared platform foundation and one pilot tenant.

This design is more expensive than a shared-database SaaS model, but it best matches the stated business goals: strong client isolation, invoice-heavy workload support, per-client database/storage tracking, enterprise readiness, and a repeatable standard architecture.

18. Appendix A: Current to Azure Service Replacement

Current	Azure Target
React/Vite SPA	Azure Static Web Apps
Supabase Auth	Microsoft Entra External ID
Supabase PostgreSQL	Azure PostgreSQL Flexible Server per client
Supabase Storage	Azure Blob Storage per client
Supabase Edge Functions	Azure Functions and Container Apps workers
pg_net	Service Bus and Event Grid
pg_cron	Timer Functions
Supabase Realtime	Azure Web PubSub / SignalR
Supabase environment secrets	Azure Key Vault
Supabase logs	Application Insights + Log Analytics
Supabase backups	PostgreSQL PITR and backup policies

19. Appendix B: CEO-Level Message

RestOps will move from a platform that is functionally strong but Supabase-centric to an Azure-native SaaS operating model. The migration gives the company a stronger enterprise story:

- Every client is isolated by design.
- Heavy invoice and payment workloads are separated per client.
- Client usage and cost become measurable.
- Shared automation keeps operations efficient.
- Terraform guarantees every client is provisioned consistently.
- Service Bus makes workflows reliable and auditable.
- Azure governance, monitoring, identity, and security controls prepare the platform for enterprise customers.

This is the right foundation for scaling RestOps from product build-out into enterprise SaaS operations.